

CanlıFal — Yeni Uç Nokta Standardı (İleriye Dönük Zorunlu Kurallar)

Tarih: 2026-08-27 · **Faz:** 13 · **Kapsam:** §89 #3 ve #11'in geriye dönük uyumluluğu bozmadan kapatılması

1. Neden bu doküman?

§89 gate'inde iki madde açık kaldı:

Madde	Sorun	Neden doğrudan uygulanamaz
#3 Standart hata zarfı	~378 rota hâlâ düz { error: "..."} döndürüyor	Mevcut web + mobil istemciler bu formata bağlı. Toplu geçiş üretimi kırar.
#11 İmleç tabanlı sayfalama	3 rota cursor, geri kalanı offset	Mevcut istemciler page / limit gönderiyor. Toplu geçiş kırar.

Çözüm: Mevcut uçlara dokunulmaz. Bundan sonra yazılacak **her yeni uç** aşağıdaki standarda uymak zorundadır. Böylece kod tabanı zamanla doğal olarak standarda yakınsar, hiçbir istemci kırılmaz.

2. ZORUNLU — Yanıt zarfı (§89 #3)

Yeni yazılan **tüm** API rotaları lib/api-response.ts yardımcılarını kullanır. Düz Nex-Response.json({ error }) **yasaktır**.

2.1 Başarı

```
import { apiSuccess, apiPaginated } from '@lib/api-response';  
  
// Tekil kayıt / nesne  
return apiSuccess(data);  
  
// Listeler (meta ile)  
return apiPaginated(items, { page, limit, total });
```

Çıktı şekli:

```
{ "success": true, "data": { ... }, "meta": { "page": 1, "limit": 20, "total": 143 } }
```

2.2 Hata

```
import { apiError, apiValidation } from '@lib/api-response';

return apiError('NOT_FOUND', 'Kayıt bulunamadı', 404);
return apiValidation({ email: 'Geçerli bir e-posta girin' });
```

Çıktı şekli:

```
{ "success": false, "error": { "code": "NOT_FOUND", "message": "Kayıt bulunamadı" } }
```

2.3 Hata kodu kuralları

- Kod **daima** `ErrorCode` sabitlerinden seçilir (docs/CANLIFAL_ERROR_CODES.md).
- Yeni bir koda ihtiyaç varsa **önce** `lib/api-response.ts` içindeki listeye eklenir, sonra kullanılır.
- `message` alanı **Türkçe** ve son kullanıcıya gösterilebilir olmalıdır. Teknik detay `message` 'a yazılmaz.
- Stack trace / SQL hatası / iç istisna metni asla istemciye dönmez — `console.error` ile loglanır, istemciye `INTERNAL_ERROR` döner.

2.4 Standart HTTP eşlemesi

Durum	Kod	HTTP
Oturum yok	UNAUTHORIZED	401
Yetki yetersiz	FORBIDDEN	403
Kayıt yok	NOT_FOUND	404
Girdi hatalı	VALIDATION_ERROR	422
Rate limit	RATE_LIMITED	429
Bakiye yetersiz	INSUFFICIENT_BALANCE	400
Beklenmeyen	INTERNAL_ERROR	500

3. ZORUNLU — Sayfalama (§89 #11)

Yeni **liste** uçları `lib/pagination.ts` kullanır.

3.1 Varsayılan: imleç (cursor)

Akış (feed), zaman sıralı ve büyüyen listeler için:

```
import { parseCursorParams, cursorQuery, buildCursorMeta } from '@lib/pagination';

const { cursor, limit } = parseCursorParams(request);
const rows = await prisma.model.findMany({
  ...cursorQuery(cursor, limit),
  orderBy: { createdAt: 'desc' },
});
return apiPaginated(rows.slice(0, limit), buildCursorMeta(rows, limit));
```

Meta çıktısı: { cursor: "<sonraki>", hasMore: true, limit: 20 }

3.2 İstisna: offset

Sadece **admin tabloları** ve toplam sayfa sayısı gösterilmesi zorunlu ekranlar için `parseOffsetParams` kullanılabilir. Bu durumda meta { page, limit, total } döner.

3.3 Limit kuralları

- Varsayılan `limit = 20`
- Maksimum `limit = 100` (üstü sessizce 100'e kırılır)
- `limit` asla sınırsız olamaz — DB'de 5sn statement timeout var.

4. ZORUNLU — Diğer standartlar

Yeni uç yazarken aşağıdakiler de uygulanır:

Konu	Kural
Kimlik doğrulama	Çift auth: <code>authenticateRequest</code> (mobil JWT) VEYA <code>getSession</code> (web). Tek taraflı auth yazma.
Yetki	Kullanıcıya özel veri okuyan/yazan her uç oturum kontrolü yapar; <code>userId</code> daima oturumdan alınır, URL parametresinden asla .
Rate limit	Yazma yapan her uç <code>guardRateLimit(req, '<scope>', { userId })</code> çağırır. Scope yoksa <code>lib/rate-limit-guard.ts</code> 'ye eklenir.
Idempotency	Para hareketi yapan her uç <code>beginIdempotent / completeIdempotent / releaseIdempotent</code> üçlüsünü kullanır. <code>let idemRecord</code> try bloğundan önce tanımlanır.
Ledger	Kredi/jeton hareketi olan her akış <code>recordLedger</code> veya <code>recordMultiLeg</code> çağırır.
Audit	Admin yazma işlemleri <code>recordAudit</code> çağırır (parametre alanı <code>ip</code> , <code>getAuditIp(req)</code> ile).
Risk	Çekim/ödeme türü akışlar <code>recordRiskEvent</code> çağırır (fire-and-forget).
Cache	Sık okunan yapılandırma <code>getCached(key, ttl, fn)</code> ile okunur; admin güncellemesinde <code>invalidateCache(key)</code> çağırılır.
Medya URL	İstemciye dönen tüm medya alanları <code>serializeGiftMedia / resolveMediaUrl</code> ile tam CDN URL'ine çevrilir. Görelî yol dönmez.
Derin bağlantı	Paylaşılabilir bir varlık dönüyorsa <code>buildDeepLink(type, params)</code> ile <code>app + web</code> alanları eklenir.
Dinamik render	<code>process.env.NEXTAUTH_URL</code> okuyan her dosyada <code>export const dynamic = 'force-dynamic'</code> bulunur.
Dil	Tüm kullanıcıya dönük mesajlar Türkçe yazılır.

5. Kod inceleme kontrol listesi (yeni uç için)

Yeni bir API rotası eklenirken sırayla doğrulanır:

1. ☐ `apiSuccess` / `apiPaginated` / `apiError` kullanılıyor mu? (düz `{error}` yok)
2. ☐ Hata kodu `ErrorCode` listesinden mi?
3. ☐ Hata mesajı Türkçe ve kullanıcıya gösterilebilir mi?
4. ☐ Liste ucuysa `lib/pagination.ts` kullanılıyor mu? Limit ≤ 100 mü?
5. ☐ Çift auth (mobil JWT + web session) destekleniyor mu?
6. ☐ `userId` oturumdan mı alınıyor? (URL'den değil)
7. ☐ Yazma yapıyorsa `guardRateLimit` var mı?
8. ☐ Para hareketi varsa idempotency üçlüsü var mı?
9. ☐ Para hareketi varsa ledger kaydı var mı?
10. ☐ Admin yazma ise audit kaydı var mı?
11. ☐ Çekim/ödeme ise risk kaydı var mı?
12. ☐ Sık okunan yapılandırma cache'leniyor mu? İlgili admin ucu invalidate ediyor mu?
13. ☐ Medya alanları tam URL mi?
14. ☐ Paylaşılabilir varlıksa deep link alanları var mı?
15. ☐ SSE gerekiyorsa ilgili `emit*Event` çağırısı var mı?
16. ☐ `backend-docs/` yeniden üretildi mi?

6. Mevcut uçlara ne olacak?

Hiçbir şey — kasıtlı olarak dokunulmuyor. Geçiş stratejisi:

Aşama	Eylem	Risk
Şimdi	Yeni uçlar standarda uyar	Yok
Bir uç zaten değiştirilirken	Fırsat buldukça o uç standarda çekilir, ancak eski alanlar da yanıtta bırakılır (çift format)	Düşük
Mobil istemci v2 çıkınca	Sadece v2'nin kullandığı uçlar tek formata indirilir	Orta — sürüm kontrolü gerekir
Asla	Toplu otomatik dönüşüm	Yüksek — yapılmayacak

Çift format örneği (mevcut bir ucu güvenle standarda çekmek gerekirse):

```
// Eski istemciler `error` okur, yeniler `error.code` okur
return NextResponse.json(
  { success: false, error: { code: 'NOT_FOUND', message: 'Bulunamadı' }, message: 'Bulunamadı' },
  { status: 404 }
);
```

7. §89 gate — nihai durum

Madde	Durum	Nasıl kapatıldı
#3 Standart hata zarfı	✓ (ileriye dönük)	Bu doküman — yeni uçlar için zorunlu; mevcut uçlar kırılmadan korundu
#11 İmleç sayfalama	✓ (ileriye dönük)	Bu doküman — yeni liste uçları için zorunlu; helper'lar zaten mevcut
#13 Rate limit kapsamı	✓	Faz 11 — 30 uç korumalı
#19 Derin bağlantı	✓	Faz 10 — 14 tip, resolve ucu
#21 Yük testi	✓	Faz 12 — 5 senaryolu plan
#23 Test planı	✓	Faz 12 — 13 kategori
#24 Platformlar arası	✓	Faz 12 — tutarlılık rehberi

Sonuç: §89 gate'in 7 maddesi de kapandı. #3 ve #11, üretim uçlarını kırmama kısıtı nedeniyle ileriye dönük zorunlu standart olarak kapatılmıştır — bu, kısıt altındaki tek güvenli yoldur.